UNIT-II

Principles of Object oriented Programming :

In order to say a programming language is object oriented it must satisfy properties of object oriented paradigm.

These are :

1. Class
2. Object
3. Data Abstraction
4. Data Encapsulation
5. Inheritance
6. Polymorphism
7. Dynamic Binding
8. Message Passing

(1) Class :

A class is grouping of objects having identical properties and common behaviour.

A class is a collection of objects of similar type.

Examples :

Class : Car
Properties : ~~speed()~~, ~~average()~~, ~~break()~~ company, model, color, capacity
Actions : speed(), average(), break()

Class : Computer
Properties : board, price, monitor type.
Actions : processing(), display().

② Objects :

Objects are variables of type class.
Objects are runtime entities of OOPs.
Objects takes up memory space and associated with address
Objects share common properties and behaviours.

Example

Object : Student
Properties : Name
DoB
Marks
Behaviours : Total()
Average()
Display().

③ Data Abstraction :

The act of representing essential ~~details~~/features without including the background details or detail explanation is called data abstraction.

Class uses the concep of abstraction. Class defines a list of abstract attributes and functions operate on these attributes.

④ Data Encapsulation :

The wrapping up of data and functions into a single unit is known as data encapsulation.

The data members is not accessible to the outside world, these functions which are wrapped in the class can access these data members.

(5) Inheritance :

The process of acquiring properties of one class ~~object~~ from another class ~~object~~ is called inheritance.

Inheritance provides the idea of reusability and reduces redundancy.

(6) Polymorphism :

Polymorphism is a Greek term which means the ability to take more than one form.

Java supports polymorphism in two ways, function/method overloading and method overriding.

(7) Dynamic Binding :

The linking of a procedure call to the code to be executed in response to the call is called dynamic binding.

It is associated with polymorphism and inheritance.

(8) Message Passing :

An object oriented program consists of a set of objects that can communicate with each other.

Objects communicate with each other through message passing.

Example :

Employee.salary(name)

↓ object ↓ message ↓ Information.

Class in Java :

In Java class contains data members and methods. Data members of a class are also known as properties, fields or attributes and methods are known as behaviour.

Syntax for defining a class :

```
class <class-name>  → Valid Java variable name.
{
    variable declaration;
    method definition;
}
```

All methods of Java must be defined inside the class only.

Example :

```
class Sum
{
    int a, b, c;
    void assign()
    {
        a=10;
        b=20;
    }
    void add()
    {
        c = a+b;
    }
    void display()
    {
        sop("val of a ="+a);
        sop("val of b = "+b);
        sop("sum ="+c);
    }
}
```

Declaring objects :

Whenever we create a class no memory is allocated for datamembers of the class.

To allocate memory for data member we need to create an object w.r.t a class.

So technicaly class has logical existance and objects have physical existance.

Definition of object :

① Instance of a class is known as objects.
② class variable is known as object.
③ Value from a class is known as object.

Syntax for creating object :

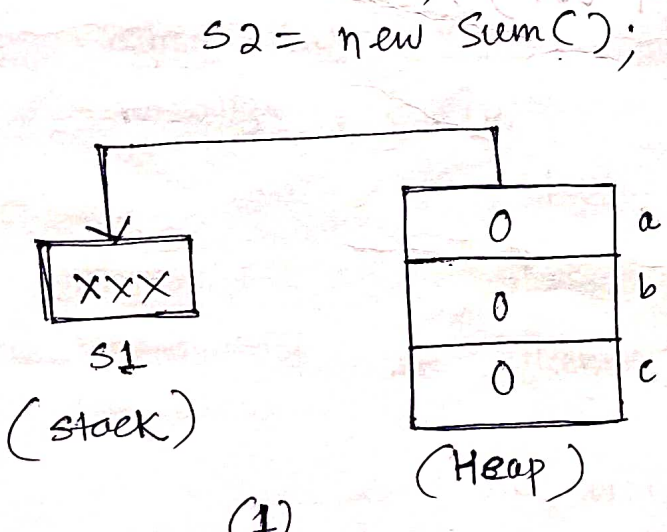
① <class-name> <objectname> = new <class-name()> ;

Sum S1 = new Sum() ;

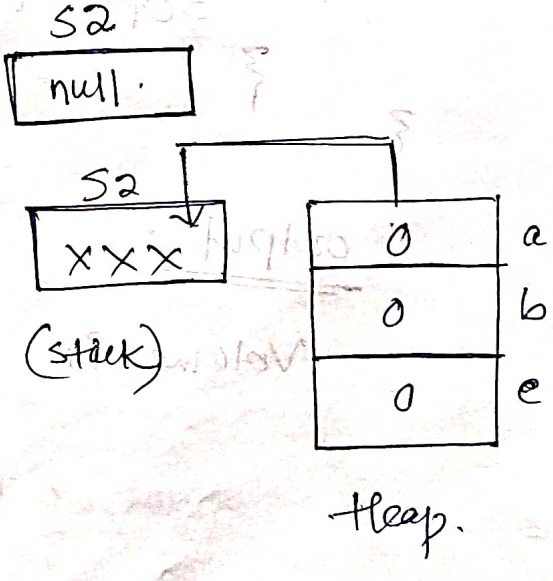
② <class-name> <object-name> ;

<object-name> = new <classname()> ;

Sum S2 ;
S2 = new Sum() ;



(1)



Function of new operator :

When we use new operator it creates memory for datamember of the class.

It takes the memory address and place it into object name.

Example :

```
class Box
{
    double width;
    double height;
    double depth;
}
class BoxDemo
{
    PSVM (String args[])
    {
        Box mybox = new Box();
        double vol;

        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

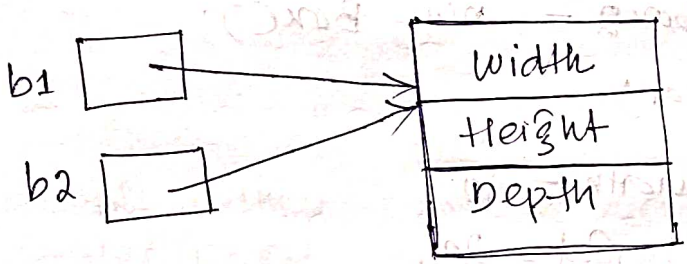
        vol = mybox.width * mybox.height * mybox.depth;
        SOP("Volume is " + vol);
    }
}
```

output :

Volume is 3000.0

Assigning object Reference Variable :

```
Box b1 = new Box();
Box b2 = b1;
```



Methods in Java :

The general form of method is :

```
type name(parameter-list)
{
    // body of method
}
```

- type specifies return type of method.
- name can be any valid java identifier name.
- Parameters are variables which stores values of argument passed to the method.

Example :

```
class Box
{
    double width;
    double height;
    double depth;

    double volume()
    {
        return width * height * depth;
    }
}
```

```
class BoxDemo4
{
    PSVM ( string args[] )
    {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        mybox1.width = 10;
        mybox2.height = 20;
        mybox1.depth = 15;

        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        vol = mybox1.Volume();

        S.O.P ("Volume is "+vol);
        vol = mybox2.Volume();
        SOP ("Volume is " + vol);
    }
}
```

output

```
Volume is 3000.0
Volume is 162.0
```

Parameterized methods in java :

* Parameters in methods used for operating on variety of data in different situations.

Example :

```
class Box
{
    double width;
    double height;
    double depth;
    double volume()
    {
        return width * height * depth;
    }
    void setDim(double w, double h, double d)
    {
        width = w;
        height = h;
        depth = d;
    }
}
class BoxDemo
{
    PSVM(String args[])
    {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;
        mybox1.setDim(10, 20, 15);
        mybox2.setDim(3, 6, 9);
        vol = mybox1.volume();
        SOP("volume is " + vol);
        vol = mybox2.volume();
        SOP("volume is " + vol);
    }
}
```

Constructors :

A constructor is a special member method called by JVM when an object is created. Constructor initializes the object.

Constructor name is same as class name.

Constructor have no return type, not even void. Because the implicit return type of constructor is the class type itself.

In java programming we have two types of constructors. Parameterized and default constructor.

A default constructor never takes any parameter.

Example :

```
class Box
{
    double width;
    double height;
    double depth;

    Box()
    {
        SOP("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }
    double volume()
    { return width * height * depth;
    }
}
```

```
class BoxDemo
{
    PSVM (String args[])
    {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        vol = mybox1.volume();
        SOP("Volume is "+vol);

        vol = mybox2.volume();
        SOP("Volume is "+vol);
    }
}
```

Output :

```
Constructing Box
Constructing Box
Volume is 1000.0
Volume is 1000.0
```

Parameterized constructor takes parameter and initializes the objects accordingly.

Example:

```
class Box
{
    double width;
    double height;
    double depth;

    Box ( double w, double h, double d)
    {
        width = w;
        height = h;
        depth = d;
    }
    double volume()
    {
        return width * height * depth;
    }
}
class BoxDemo
{
    PSVM ( String args[])
    {
        Box b1 = new Box(10,20, 30);
        Box b2 = new Box( 3,6, 9);

        double vol;
        vol = b1.volume();
        SOP( " volume is " + vol);

        vol = b2.volume();
        SOP ( " volume is " + vol);
    }
}
```

Constructor overloading :

A constructor is said to be overloaded if and only if constructor name is same but signature is different. Signature represents - No of parameters, Type of parameter and order of parameter.

```
Box b1 = new Box(10,20);
Box b2 = new Box(10);
Box b3 = new Box(10.75, 100.65);
Box b4 = new Box(10, 10.75);
Box b5 = new Box(10.75, 10);
```

Object Parameterized Constructor :

An object parameterized constructor is one which takes object as an parameter.

The purpose of parameterized constructor is to copy the content of one object into other object where both the objects belongs to same type.

```
Box b6 = new Box(10,20,30);
Box b7 = new Box(b6);
```

Example :

```
class Box
{
    double width;
    double height;
    double depth;

    Box(Box ob)
    {
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
```

```
Box ( double w, double h, double d)
{
    width = w;
    height = h;
    depth = d;
}

Box()
{
    width = -1;
    height = -1;
    depth = -1;
}

Box ( double len)   // Box as a cube.
{   width = height = depth = len;
}
double volume()
{
    return width * height * depth;
}

class OverloadConst
{   PSVM ( String args[])
    {
        Box mybox1 = new Box (10, 20, 15);
        Box mybox2 = new Box();
        Box mybox3 = new Box(7);
        Box myclone = new Box(mybox1);
        double vol;
        vol = mybox1.volume(); SOP ("Volume is" + vol);
        vol = mybox2.volume(); SOP ("Volume is" + vol);
        vol = mybox3.volume(); SOP("Volume of cube is" + vol);
        vol = myclone.volume(); SOP ("Volume of clone is" + vol);
    }
}
```

The "this" Keyword :

"this" in java is used to refer current object inside any method of a class.

Whenever the formal parameters and data members of the class are similar there is a possibility that ambiguity may arise.

To differentiate formal parameters and data members of the class we write this before the data members.

Example :

```
Box (double width, double height, double depth)
{
    this.width = width;
    this.height = height;
    this.depth = depth;
}
```

The this() call can be used to invoke the current class constructor.

Calling default constructor from parameterized constructor :

```
class A
{
    A() { SOP("Hello A"); }
    A(int x) { this();
               SOP(x);
             }
}
class TestThis
{
    PSVM(String args[])
    { A a = new A(10);
    }
}
```

Output : Hello A
10

Calling parameterized constructor from default constructor :

```
class A
{
    A() { this(5);
          SOP("Hello A");
        }
    A(int x) { SOP(x);
             }
}
class TestThis2
{
    PSVM(String args[])
    { A a = new A();
    }
}
```

Output : 5
Hello A

Overloading Methods :

Declaring two or more methods with the same name but with different parameters is called method overloading.

It is one of the ways java supports polymorphism.

The overloaded methods must be different in the type and number of their parameters

When we call an overloaded method, it simply executes the method whose parameter match the arglements.

Example :

```
class overloaded
{
    void test()
    { SOP("No parameters");
    }
    void test(int a)
    { SOP("a:"+a);
    }
    void test(int a, int b)
    { SOP("a and b:"+a+" "+b);
    }
    double test(double a)
    {
        SOP("double a:"+a);
        return a*a;
    }
class Test
{
    PSVM(String args[])
    { overloaded ob = new overloaded();
      double result;
      ob.test();
      ob.test(10);
      ob.test(10,20);
      result = ob.test(123.25);
      SOP("Result is "+result);
    }
}
```

Types of datamembers in Java :

A class in java contains data members and methods. Data members of the class are classified into two types : instance datamembers and static data members.

Instance Data members

(1) Memory is created each and every time an object is created

(2) For each object value is specific to that object

(3) syntax :
datatype variable1, ...
variablen;

(4) Instance data members must be accessed with respect to objects.
objectname. instancedatamember;

(5) Values of instance data-member are not shareable.

(6) All instance data member are known as object level variable.

int pin, stdroll;

Static Data Members.

(1) Memory space is created only once when class is loaded.

(2) Value is same for all objects.

(3) Syntax :
static datatype variable1, ...
variable n;

(4) Static data members must be accessed with respect to class name.
classname. static data member;

(5) All static data member values are shareable.

(6) All static datamembers are known as class level datamembers.

static String manager;
static String collegename;

Types of methods in Java :

Each and every class of Java contains collection of methods. The methods of Java are divided into two types. Instance methods. and static methods.

Instance Methods

(1) Instance methods are used to perform repeated operations.

(2) returntype methodname (list of parameters)
{
Block of statements;
}

(3) Instance method in Java must be accessed with object name.

object.instancemethod();

(4) Instance methods are also known as object level methods.

Static Methods

(1) Static methods are those designed to perform onetime operations.

(2) static returntype methodname (list of parameters)
{
Block of statements;
}

(3) Each and every static method must be accessed w.r.t class name.

classname.staticmethod();

(4) Static methods are also known as class level methods.

Rules for static and instance ~~case~~ method :

(1) static method :

(1) Static method can call another static method within the class directly and outside the class through the class name.

(2) Static method can call a non-static method within the class through the object name and outside the class through the object name.

(3) one static method invoke the static variable within the class directly but outside the class through the class name.

(4) one static method invoke instance variable within the class through the object name and outside the class through the object name.

(2) Instance methods :

(5) one non-static method invoke another instance (non-static) method within class directly but outside the class through the object name.

(6) one non-static method invoke static method within the class directly but outside the class through the class name.

(7) one instance method invoke instance variable within the class directly outside the class through the object name.

(8) one instance method invoke static variable within the class directly but outside the class through the class name.

The finalize() method :

Sometimes an object will need to perform some action when it is destroyed.

By ~~using~~ using final method we can define specific actions that will occure when an object is freed by garbage collector.

The general form of finalize() method is :

```
protected void finalize() throws Throwable
{
    // finalization code here
}
```

finalize() is only called just before garbage collection. But not when an object goes out of scope.

So you cannot know when finalize() will be executed.

Garbage Collection :

In java objects are destroyed automatically by garbage collector.

When no reference to an object exists, then that object is assumed to be no longer needed and the memory occupied by the object is released by the garbage collector.

Garbage collection occures periodically during execution of java program

It will not occure simply because one or more object exists but are not used.

You should not think about garbage collection while writing your program.